

Algoritmos II

Árvores binárias de busca - Continuação

Prof. Rodrigo Minetto

rminetto@dainf.ct.utfpr.edu.br

Universidade Tecnológica Federal do Paraná

Árvores binárias de busca - Remoção

A operação de **remoção** é mais complexa que a inserção. Novamente, a implementação dessa função vem da natureza recursiva da árvore.

Árvores binárias de busca - Remoção

Se a árvore **for vazia**, nada tem de ser feito, pois o elemento não está presente na árvore.

Árvores binárias de busca - Remoção

Se a árvore **não for vazia**, comparamos o valor armazenado no nó raiz ao valor que se deseja retirar da árvore. Se o valor associado à raiz for **maior** do que o valor a ser retirado, chamamos a função recursivamente para retirar o elemento da subárvore à **esquerda**.

Árvores binárias de busca - Remoção

Se o valor associado à raiz for **menor** do que o valor a ser retirado, chamamos a função recursivamente para retirar o elemento da subárvore à **direita**.

Árvores binárias de busca - Remoção

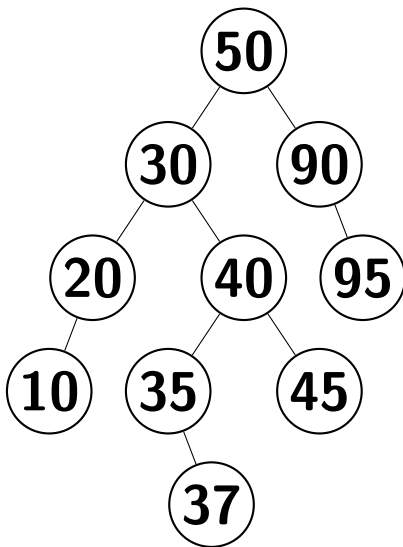
Finalmente, se o valor for **igual**, então simplesmente o **removemos** da árvore. Portanto, sempre iremos remover um nó raiz de uma (sub)árvore. No entanto, existem **três** situações possíveis.

Árvores binárias de busca - Remoção

A primeira, e mais simples, é quando se deseja **retirar** uma raiz que é **folha** (isto é, uma raiz que não tem filhos). Nesse caso, basta liberar a memória alocada pelo elemento e ter como valor de retorno a raiz atualizada, que passa a ser NULL.

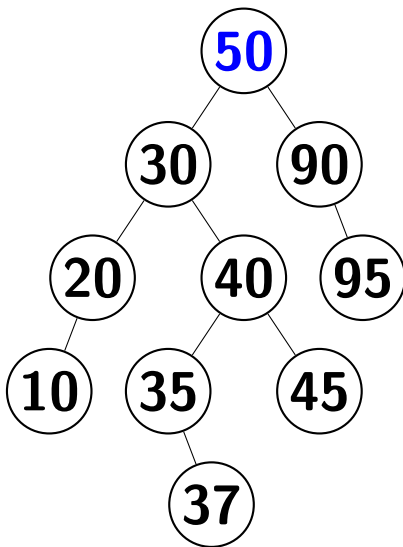
Árvores binárias de busca - Remoção

Remover o elemento '10'



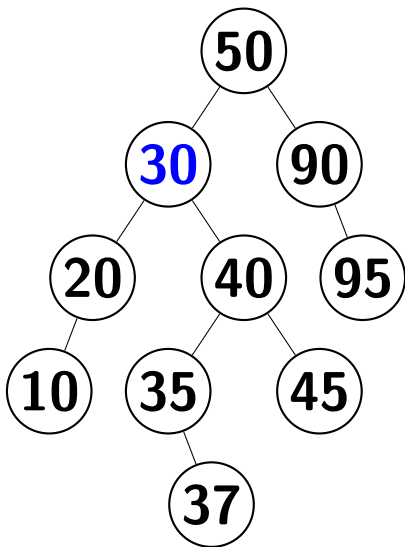
Árvores binárias de busca - Remoção

Remover o elemento '10'



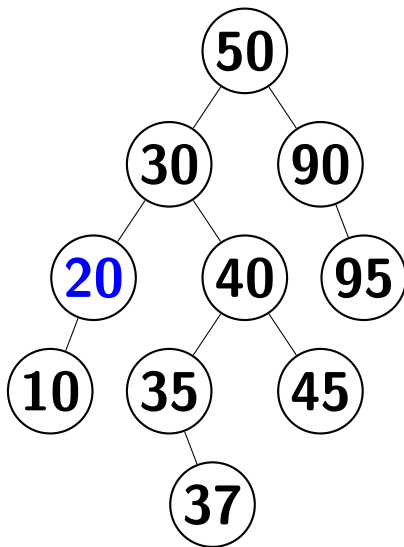
Árvores binárias de busca - Remoção

Remover o elemento '**10**'



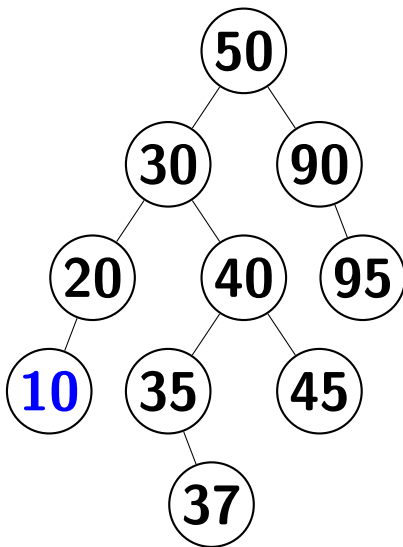
Árvores binárias de busca - Remoção

Remover o elemento '**10**'



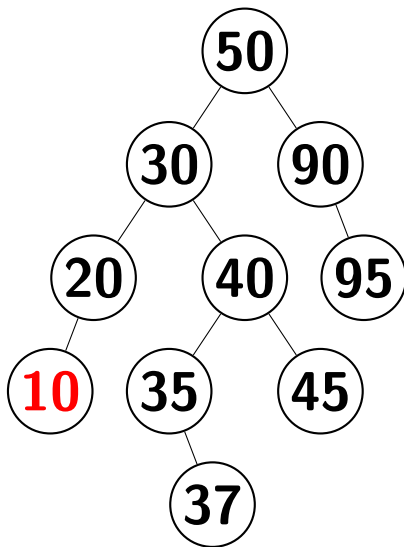
Árvores binárias de busca - Remoção

Remover o elemento '**10**'



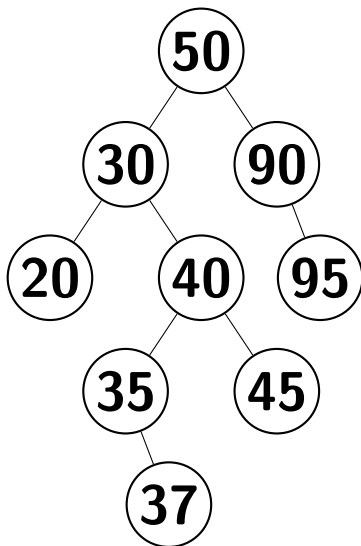
Árvores binárias de busca - Remoção

Remover o elemento '**10**'



Árvores binárias de busca - Remoção

Remover o elemento '10'

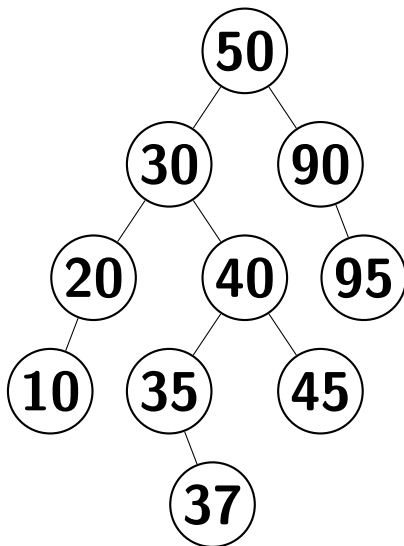


Árvores binárias de busca - Remoção

A segunda situação, ainda simples, acontece quando a raiz a ser retirada possui um único filho. Ao se retirar esse nó, o filho toma o lugar da raiz.

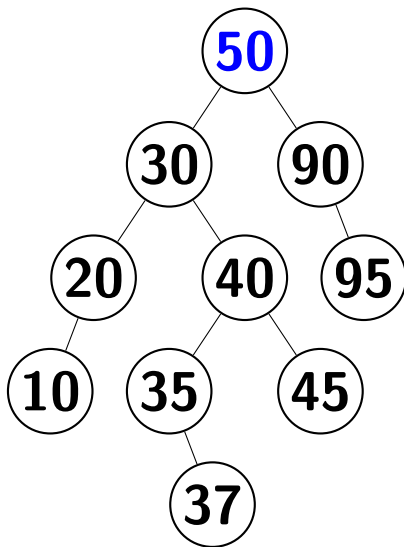
Árvores binárias de busca - Remoção

Remover o elemento '90'



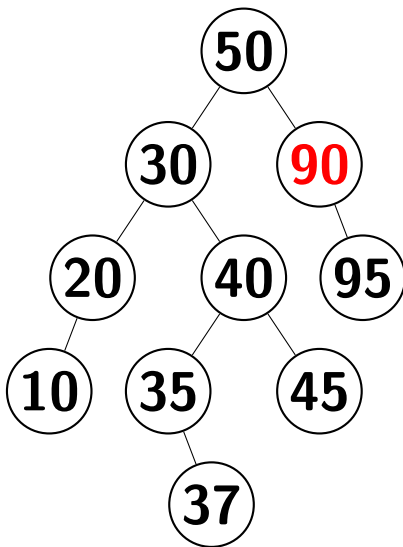
Árvores binárias de busca - Remoção

Remover o elemento '90'



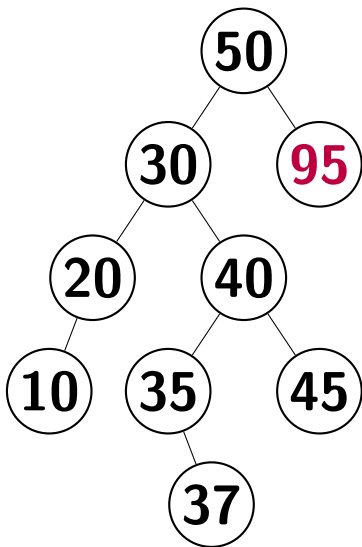
Árvores binárias de busca - Remoção

Remover o elemento '90'



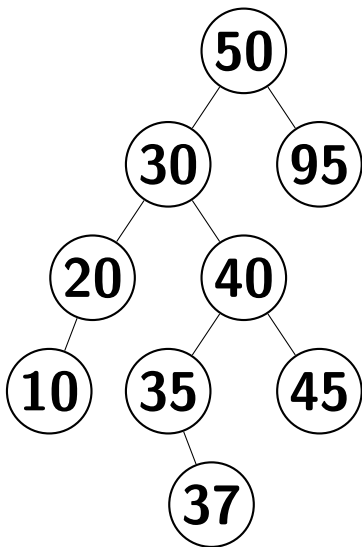
Árvores binárias de busca - Remoção

Remover o elemento '90'



Árvores binárias de busca - Remoção

Remover o elemento '90'



Árvores binárias de busca - Remoção

A terceira situação, a mais complicada, acontece quando a raiz a ser retirada tem dois filhos. Para poder retirar esse nó da árvore, devemos proceder da seguinte forma:

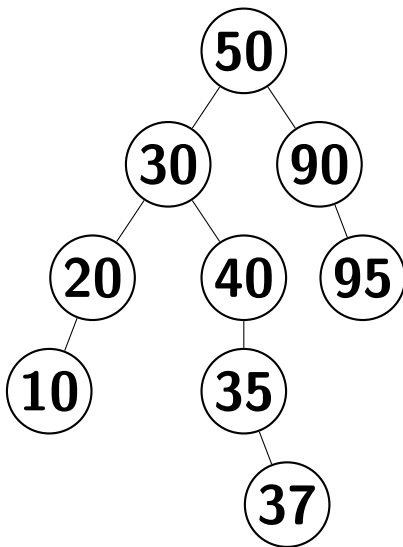
- **Encontre** o elemento que precede a raiz (que se deseja retirar) na ordenação. Isso equivale a encontrar o elemento mais à direita da subárvore à esquerda da raiz.

Árvores binárias de busca - Remoção

- **Troque** a informação da raiz com a informação do nó encontrado e **retire-o** da sub(árvore) à esquerda. Note que após a troca, a remoção desse nó é trivial, pois, ou ele é um nó folha ou um nó com apenas um filho (um filho à esquerda).

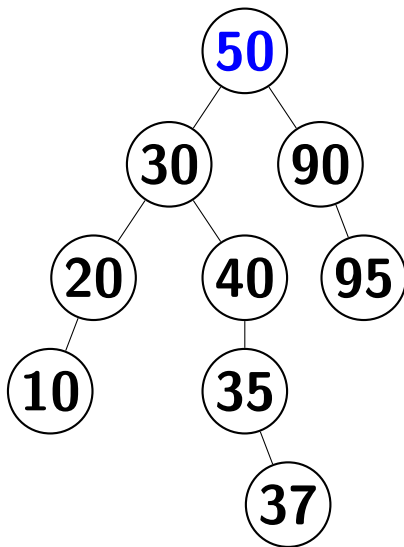
Árvores binárias de busca - Remoção

Remover o elemento '50'



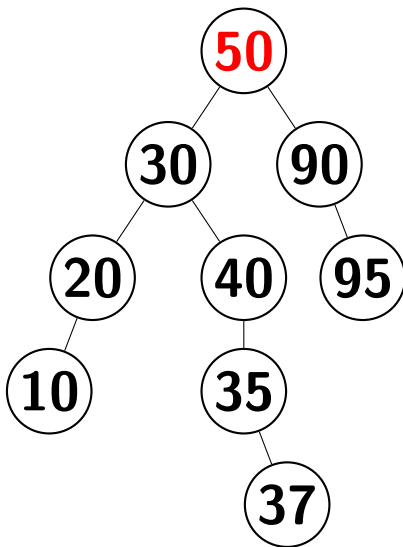
Árvores binárias de busca - Remoção

Remover o elemento '50'



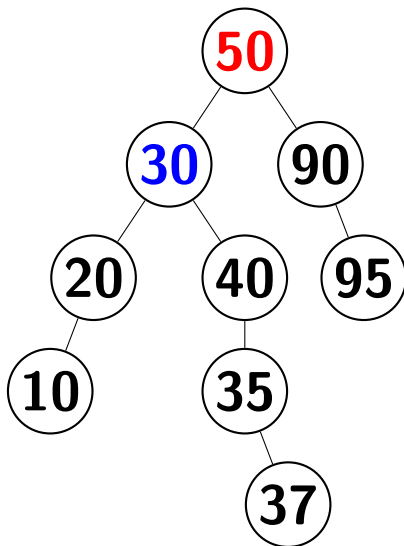
Árvores binárias de busca - Remoção

Remover o elemento '50'



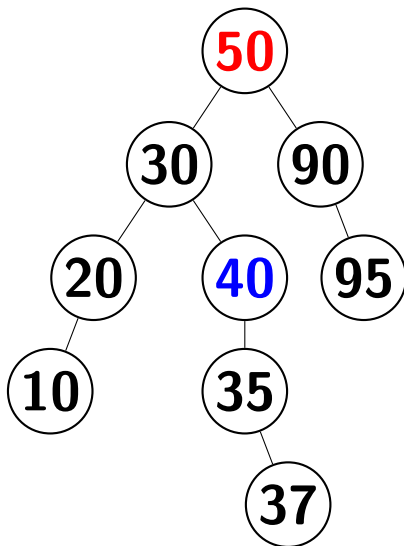
Árvores binárias de busca - Remoção

Remover o elemento '50'



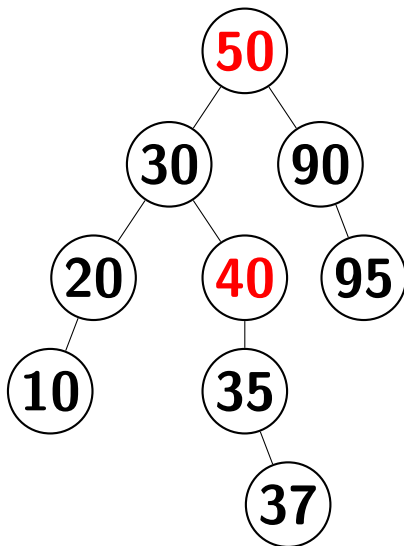
Árvores binárias de busca - Remoção

Remover o elemento '50'



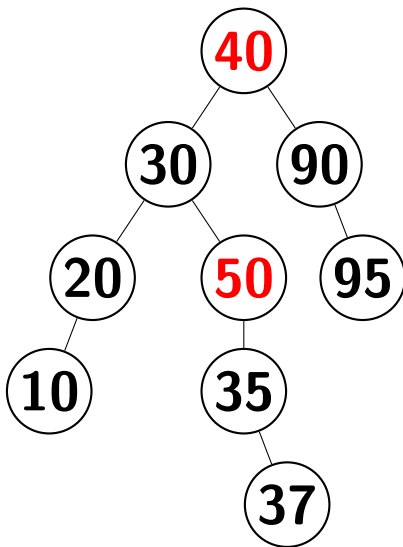
Árvores binárias de busca - Remoção

Remover o elemento '50'



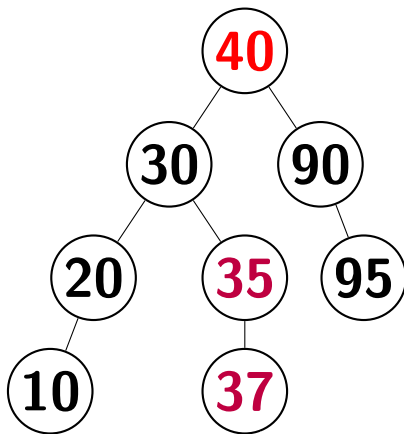
Árvores binárias de busca - Remoção

Remover o elemento '50'



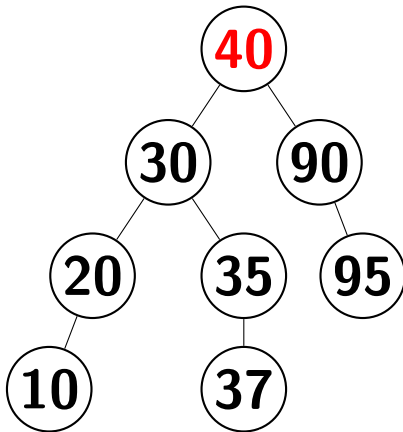
Árvores binárias de busca - Remoção

Remover o elemento '50'



Árvores binárias de busca - Remoção

Remover o elemento '50'



Árvores binárias de busca - Remoção

O procedimento descrito acima deve ser seguido para não haver **violação** da ordenação da árvore. Vale notar, que assim como foi escolhido o nó mais à direita da subárvore à esquerda, também poderíamos ter escolhido o nó mais à esquerda da subárvore à direita (o nó que segue a raiz na ordenação).

Exercício) Escreva uma função para remover elementos em uma **árvore binária de busca**. Considere todos os casos existentes. Ou seja, teste sua função de remoção para os três casos vistos na aula.